

Лабораторна робота

Дослідження швидкодії операції `std::count_if`

Варіант 4 - `count_if`

Виконав: Дмитро Демчук, група К-25
2026

1. Завдання

Дослідити швидкодію операції `std::count_if` над послідовностями різного обсягу: (1) без політики виконання, (2) з різними політиками виконання (`seq`, `par`, `par_unseq`), (3) власним паралельним алгоритмом, що розбиває дані на K частин, обробляє кожну в окремому потоці послідовним `count_if`, а результат зводить `std::reduce`. Дослідити залежність часу від K , знайти найкраще K та порівняти його з кількістю апаратних потоків. Дослідження виконати в режимі `RELEASE` з двома рівнями оптимізації: без оптимізації (`-O0`) та з максимальною (`-O3`).

2. Методика

2.1. Компілятор

GCC (Homebrew `g++-16`), стандарт C++20. На macOS компілятор за замовчуванням - Apple Clang, але його стандартна бібліотека `libc++` не підтримує політики виконання `std::execution::par/seq/unseq/par_unseq` - код з ними не компілюється. Тому збірка виконана через GCC, чия бібліотека `libstdc++` реалізує ці політики через Intel TBB.

Команди збірки (Makefile):

```
g++-16 -std=c++20 -I<tbb>/include -L<tbb>/lib -ltbb -pthread -O0 lab.cpp -o lab_00
g++-16 -std=c++20 -I<tbb>/include -L<tbb>/lib -ltbb -pthread -O3 lab.cpp -o lab_03
```

2.2. Дані

Тестові послідовності - випадкові цілі числа в діапазоні $[2, 100000]$, згенеровані `std::mt19937` (`seed=42`) та `std::uniform_int_distribution`, збережені заздалегідь у текстові файли `data/seq_N.txt` окремою програмою `generate.cpp`. Досліджено три обсяги: $N = 100\,000$; $1\,000\,000$; $10\,000\,000$. Усі режими обробки використовують один і той самий набір файлів.

2.3. Предикати

Використано два предикати різної «ваги»: `isEvenNumber` ($x \% 2 == 0$) - дешевий, одна арифметична операція; `isPrime` - дорогий, перевірка простоти пробними діленнями до $\sqrt{x}$.

2.4. Вимірювання часу

Для кожного дослідження фіксуються два таймери (`std::chrono::high_resolution_clock`): `total` - читання файлу з диска, завантаження в RAM, обчислення та вивід результату разом; `compute` - лише сам виклик `count_if` на вже завантажених у RAM даних. Різниця між ними показує, яку частку часу займає файловий ввід/вивід, а не сам алгоритм.

3. Частина 1-2: бібліотечний count_if - без політики та з політиками

Час обчислення (compute, мс) для кожного режиму. Результат (кількість елементів, що задовольняють предикат) співпав у всіх режимах для однакового N і предиката - це підтверджує коректність усіх варіантів.

N	предикат	none	seq	par	par_unseq
100 000	even	0.174	0.174	0.049	0.090
100 000	prime	3.668	3.242	0.662	0.767
1 000 000	even	0.955	0.936	0.173	0.173
1 000 000	prime	26.670	26.500	5.930	5.943
10 000 000	even	9.430	9.407	1.858	1.831
10 000 000	prime	266.490	265.122	60.116	58.846

Таблиця 1. Час обчислення (-O3), мс.

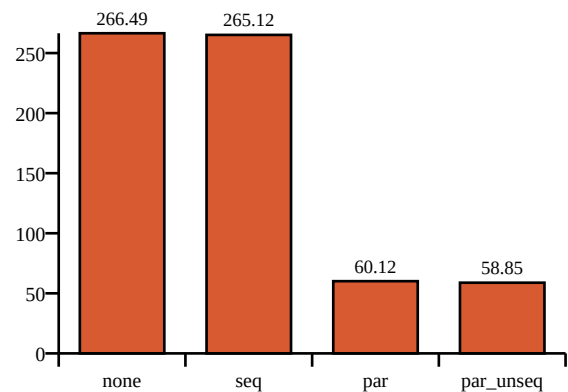
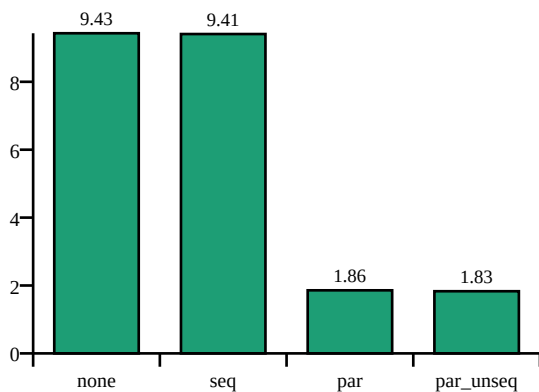


Рис. 1-2. Час обчислення на N=10 000 000 (-O3): зліва - дешевий предикат even, справа - дорогий предикат prime.

Висновок: на дешевому предикаті (even) виграш par/par_unseq над seq помітний, але накладні витрати на розподіл роботи між потоками «з'їдають» значну частку теоретичного прискорення. На дорогому предикаті (prime) прискорення набагато виразніше - приблизно у 4.4 рази (265.1 мс -> 60.1 мс) на 8 апаратних потоках, бо самої роботи, яку є що розпаралелювати, значно більше відносно накладних витрат. Різниця seq проти «без політики» - в межах похибки вимірювання: обидва варіанти фактично однопотокові.

4. Частина 3: власний паралельний алгоритм

Масив ($N=10\,000\,000$, предикат `prime`) ділиться на K приблизно однакових частин; кожна частина обробляється в окремому `std::thread` послідовним `count_if`, а часткові результати зводяться `std::reduce`. `hardware_concurrency()` на тестовій машині = 8.

K	compute, мс (-O3)	compute, мс (-O0)
1	264.957	714.917
2	136.968	368.792
4	71.459	191.928
6	71.509	160.748
8	64.217	130.639
12	61.040	128.491

Таблиця 2. Залежність часу від K .

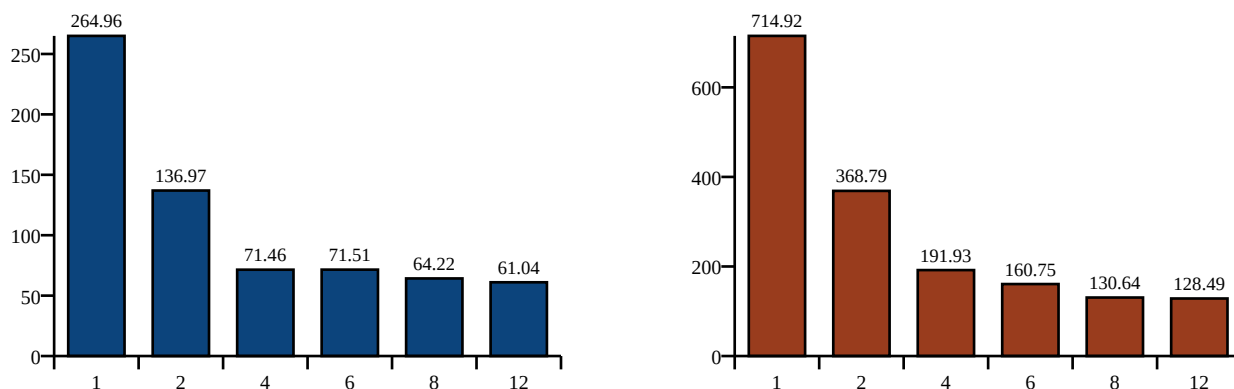


Рис. 3-4. Час власного паралельного алгоритму залежно від K .

Найкраще знайдене значення: $K = 12$ (61.04 мс на -O3, 128.49 мс на -O0) - в обох збірках мінімум серед перевірених K . Це у 1.5 рази більше за `hardware_concurrency()` = 8.

Закон зростання: від $K=1$ до $K=4$ час падає майже обернено пропорційно K (подвоєння $K \approx$ вдвічі менший час - 265→137→71 мс), тобто прискорення тут близьке до лінійного. Після $K \approx 6$ крива виходить на плато: подальше збільшення K до 8 і 12 дає лише невеликий додатковий виграш (71→64→61 мс). Це узгоджується із законом Амдала - після того як кількість потоків наближається до кількості реальних апаратних виконавців, додаткові потоки лише ділять той самий обсяг паралельних ресурсів, а не додають нову обчислювальну потужність; невеликий подальший виграш при $K=12$ (понад `hardware_concurrency`) пояснюється тим, що частини стають достатньо малими, аби краще заповнити паузи (наприклад, коли операційна система ненадовго перемикає інший потік на те саме ядро), а накладні витрати на створення потоків усе ще незначні відносно обсягу роботи кожного потоку.

5. Вплив рівня оптимізації (-O0 проти -O3)

Порівняння тієї самої операції (N=10 000 000, predicate=prime, без політики) зібраної без оптимізації та з максимальною:

Режим	-O0, мс	-O3, мс	прискорення
без політики (compute)	712.659	266.490	2.67x
seq (compute)	731.684	265.122	2.76x
par (compute)	127.546	60.116	2.12x
власний алгоритм, K=8 (compute)	130.639	64.217	2.03x

Таблиця 3. Вплив рівня оптимізації.

Сам лише рівень оптимізації компілятора дає прискорення у 2-2.8 рази - без жодної зміни коду і без жодного додаткового потоку. Це важливо враховувати: якби послідовну версію збирали з -O0, а паралельну - з -O3, виміряне «прискорення від паралелізму» було б завищене, бо частина виграшу насправді походила б від оптимізації компілятора, а не від розподілу роботи між потоками. Тому коректне порівняння політик виконання проведено в межах одного рівня оптимізації (окремо для -O0 і окремо для -O3).

6. Висновки

- Бібліотечні політики виконання (par, par_unseq) дають суттєве прискорення count_if лише тоді, коли предикат достатньо «важкий» (isPrime): до ~4.4x на 8 апаратних потоках. На дешевому предикаті (isEvenNumber) виграш є, але значно скромніший - накладні витрати розподілу роботи стають відносно вагомими.
- Різниця між «без політики» і seq несуттєва - обидва варіанти виконуються одним потоком.
- Власний паралельний алгоритм (розбиття на K потоків + std::reduce) поводить ся так само, як бібліотечний par: прискорення близьке до лінійного, поки K не наблизиться до кількості апаратних потоків (8), після чого крива виходить на плато. Найкраще знайдене K=12, тобто в 1.5 рази більше за hardware_concurrency().
- Рівень оптимізації компілятора (-O0 проти -O3) впливає на швидкість не менше, ніж паралелізм: сам по собі дає 2-2.8x прискорення. Дослідження політик і власного алгоритму коректно проводити лише в межах одного рівня оптимізації.

Повний вивід програми для обох рівнів оптимізації наведено у файлах results/output_O0.txt та results/output_O3.txt, що додаються до звіту разом з вихідним кодом (generate.cpp, lab.cpp, Makefile).

7. Додаток: скріншоти

7.1. Збирання виконаного файлу

```
king@Dmytro-MacBook-Pro lr_2 % make run
/opt/homebrew/bin/g++-16 -std=c++20 -I/opt/homebrew/opt/tbb/include -L/opt/homebrew/opt/tbb/lib -ltbb -pthread -O0 lab.cpp -o lab_00
/opt/homebrew/bin/g++-16 -std=c++20 -I/opt/homebrew/opt/tbb/include -L/opt/homebrew/opt/tbb/lib -ltbb -pthread -O3 lab.cpp -o lab_03
./generate
generated data/seq_100000.txt
generated data/seq_1000000.txt
generated data/seq_10000000.txt
./lab_00 > results/output_00.txt
./lab_03 > results/output_03.txt
```

Рис. 5. Результат make run: збірка -O0/-O3 та генерація даних без помилок.

7.2. Результат роботи програми

```
/Users/king/Desktop/lr_2/lab_03
hardware_concurrency = 8

N=100000 predicate=even policy=none result=50016 total=3.443ms compute=0.103ms
N=100000 predicate=even policy=seq result=50016 total=3.188ms compute=0.101ms
N=100000 predicate=even policy=par result=50016 total=3.734ms compute=0.1ms
N=100000 predicate=even policy=par_unseq result=50016 total=3.098ms compute=0.042ms
N=100000 predicate=prime policy=none result=9435 total=5.957ms compute=2.763ms
N=100000 predicate=prime policy=seq result=9435 total=6.192ms compute=2.623ms
N=100000 predicate=prime policy=par result=9435 total=3.647ms compute=0.697ms
N=100000 predicate=prime policy=par_unseq result=9435 total=3.446ms compute=0.587ms

N=1000000 predicate=even policy=none result=500203 total=30.403ms compute=0.938ms
N=1000000 predicate=even policy=seq result=500203 total=28.914ms compute=0.943ms
N=1000000 predicate=even policy=par result=500203 total=28.134ms compute=0.165ms
N=1000000 predicate=even policy=par_unseq result=500203 total=28.239ms compute=0.191ms
N=1000000 predicate=prime policy=none result=96284 total=54.891ms compute=26.665ms
N=1000000 predicate=prime policy=seq result=96284 total=54.479ms compute=29.486ms
N=1000000 predicate=prime policy=par result=96284 total=34.173ms compute=6.579ms
N=1000000 predicate=prime policy=par_unseq result=96284 total=34.27ms compute=6.432ms
```

Рис. 6. Вивід lab_03 - частина 1-2 (N=100 000 та N=1 000 000).

```
N=10000000 predicate=even policy=none result=4998689 total=298.74ms compute=9.44ms
N=10000000 predicate=even policy=seq result=4998689 total=289.91ms compute=9.467ms
N=10000000 predicate=even policy=par result=4998689 total=281.595ms compute=1.648ms
N=10000000 predicate=even policy=par_unseq result=4998689 total=282.578ms compute=1.688ms
N=10000000 predicate=prime policy=none result=958677 total=545.679ms compute=267.716ms
N=10000000 predicate=prime policy=seq result=958677 total=545.524ms compute=265.404ms
N=10000000 predicate=prime policy=par result=958677 total=338.97ms compute=59.72ms
N=10000000 predicate=prime policy=par_unseq result=958677 total=340.258ms compute=58.879ms

custom parallel count_if
N=10000000 predicate=prime
K=1 result=958677 compute=265.637 ms
K=2 result=958677 compute=137.345 ms
K=4 result=958677 compute=73.494 ms
K=6 result=958677 compute=71.811 ms
K=8 result=958677 compute=62.782 ms
K=12 result=958677 compute=64.588 ms
```

Рис. 7. Вивід lab_03 - частина 1-2 (N=10 000 000) та частина 3 (K).